

DATABRICKS MILESTONE 2

Trainee Preparation & Practice Notes

Java | Scala | Spark RDDs | Spark DataFrames | Advanced Spark | Streaming | JDBC & Concurrency

Purpose: A detailed assessment-preparation handbook built from the topics and depth represented in the finalized Milestone 2 MCQ bank. Use it to revise concepts, syntax, common traps, and scenario reasoning before the assessment.

Prepared for trainee self-study and guided revision

How to Use This Handbook

Milestone 2 is not a memory-only assessment. The question bank mixes direct concepts, code interpretation, basic application, and a smaller number of stretch scenarios. Preparation should therefore combine concept revision with small runnable examples and the ability to explain why one option is better than another.

- **Pass 1 - Understand:** Read the concept sections and make sure you can explain each term in your own words.
- **Pass 2 - Execute:** Run the small Java/Scala/Spark snippets. Change one value and predict the result before executing.
- **Pass 3 - Compare:** Practice differences such as RDD vs DataFrame, map vs flatMap, stream vs batch, overloading vs overriding, and stack vs heap.
- **Pass 4 - Scenario reasoning:** For Advanced Spark, Streaming, JDBC and concurrency, identify the problem first, then choose the mechanism that directly addresses it.
- **Pass 5 - Final revision:** Use the chapter checklists and practice drills. Revisit only the items you cannot explain without looking at the notes.

Assessment Coverage Map

#	Assessment Domain	MCQs in Bank	Difficulty Pattern	Depth Pattern
1	Java - Fundamentals, Core, Collections	16	Easy: 16	Basic: 16
2	Java - OOPs	16	Easy: 16	Basic: 16
3	Java - File I/O, APIs	12	Easy: 4, Medium: 8	Basic: 4, Basic Application: 8
4	Java - OOPs/APIs/Collections	8	Medium-High / Stretch: 8	Stretch: 8
5	Scala Basics [Data Structures]	12	Medium: 12	Basic Application: 12
6	Scala - transformations	8	Medium: 8	Medium Application: 8
7	Scala - RDDs	16	Medium: 16	Basic Application: 16
8	Scala - File I/O	8	Medium: 8	Basic Application: 8
9	Scala - dataframes	8	Medium: 8	Medium Application: 8
10	Advanced Spark (Execution Plans / UDFs / External Sources)	8	Medium: 8	Medium Application: 8
11	Spark Streaming, Fault Tolerance, Data Processing	8	Medium-High / Stretch: 8	Stretch: 8

Important: Question counts show emphasis in the practice bank, not guaranteed marks in one assessment set. Use them to prioritize revision, not to skip smaller topics.

How Milestone 2 Questions Are Framed

- **Concept recall:** Know definitions, purposes and basic differences without needing deep implementation detail.
- **Code interpretation:** Read short Java/Scala snippets and predict which constructor, method, collection behavior or expression result applies.
- **Basic application:** Choose the right API or Spark operation for a straightforward requirement.
- **Scenario reasoning:** For Spark execution, streaming, JDBC and concurrency, identify the direct technical problem before choosing the mechanism.

Depth Boundary - What You Do Not Need to Overprepare

- JVM garbage-collector algorithms, bytecode instruction details or deep memory-layout internals.
- Advanced Java concurrency implementation internals, lock-free algorithms or production thread-pool tuning.
- Advanced JDBC pool sizing, isolation-level theory or database-specific transaction internals.
- Spark scheduler internals, Catalyst rule internals, code generation internals or detailed shuffle-file architecture.
- Structured Streaming state-store internals or complex watermark edge cases beyond the basic late-data concept.
- Advanced Scala type-system topics such as type classes/implicit unless explicitly revised separately by the trainer.

Priority	What to master
High	Java fundamentals/OOP, Scala basics, RDD transformations/actions, pair RDDs, persistence, core DataFrame/schema reasoning.
Medium	Java I/O/APIs, Scala file handling, functional transformations, Spark execution plans/UDF/JDBC integration.
Stretch	Streaming recovery/windows, concurrency, deadlocks/races, CompletableFuture, ConcurrentHashMap, CopyOnWriteArrayList, HikariCP, JDBC rollback scenarios.

1. Java Fundamentals, Core Java and Collections

Assessment emphasis: 16 MCQs in the bank. The questions are mainly Easy/Basic. You are expected to recognize core runtime behavior, collection choices, generics, iteration, numeric/operator rules, and serialization basics.

JVM and Bytecode Execution

Concept. Java source code is compiled into bytecode. The JVM executes that bytecode and provides the runtime environment, memory management and platform abstraction.

What you should know for the assessment. Know the role of JDK, JRE and JVM; understand that javac compiles .java files into .class bytecode and the JVM executes bytecode. Do not confuse the JVM with the Java compiler.

```
javac App.java
java App
```

Common traps. JVM does not normally compile Java source files directly. JDK contains development tools; JVM is the execution engine.

Preparation focus. Be able to identify the correct component in a short scenario: compile -> javac/JDK; execute bytecode -> JVM.

Stack and Heap Memory

Concept. Each method call has a stack frame containing local variables and call information. Objects and arrays are generally allocated on the heap and referenced from variables.

What you should know for the assessment. For assessment purposes, use the beginner mental model: local primitives/references belong to a method stack frame; created objects live on the heap; multiple references can point to the same heap object.

```
Person p = new Person(); // p is a reference; the Person object is on the heap
```

Common traps. Do not say all variables are on the heap or all objects are on the stack. Avoid overcomplicating JVM implementation details beyond the taught model.

Preparation focus. Practice drawing a simple memory picture for two references pointing to one object.

Wrapper Classes and Autoboxing

Concept. Wrapper classes represent primitive values as objects, for example Integer for int and Double for double. Autoboxing converts primitives to wrappers automatically; unboxing converts wrappers back to primitives.

What you should know for the assessment. Know common wrappers and why collections such as List<Integer> use reference types rather than primitive type parameters.

```
Integer x = 10; // autoboxing
int y = x;      // unboxing
```

Common traps. Wrapper equality with == can be misleading; for value comparison prefer equals() when objects are involved.

Preparation focus. Know primitive-wrapper pairs and identify autoboxing/unboxing in short snippets.

Static vs Instance Members

Concept. A static member belongs to the class, while an instance member belongs to each object.

What you should know for the assessment. Know how to access static members using the class name and instance members using an object. Understand that static methods cannot directly access instance fields without an object reference.

```
class Counter {
  static int total;
  int id;
}
Counter.total++;
```

Common traps. Do not assume static means constant. final controls reassignment; static controls class-level ownership.

Preparation focus. Practice deciding whether data should be shared by all objects or stored separately per object.

Collection Interfaces: List, Set and Map

Concept. List keeps an ordered sequence and permits duplicates; Set models uniqueness; Map stores key-value associations.

What you should know for the assessment. Be able to choose ArrayList/List for ordered duplicates, Set for unique items, and Map for lookups by key.

```
List<String> names = new ArrayList<>();
Set<String> ids = new HashSet<>();
Map<Integer,String> users = new HashMap<>();
```

Common traps. A Map is not a subtype of Collection. A Set does not promise indexed access like a List.

Preparation focus. For every scenario, ask: order? duplicates? key-value lookup?

HashSet Equality Semantics

Concept. HashSet uses hashCode() and equals() to determine logical uniqueness of objects.

What you should know for the assessment. Know that custom objects should implement equals() and hashCode() consistently when logical duplicates must be detected.

Common traps. Two different object instances can still be considered equal. If equals() is overridden without a compatible hashCode(), hash-based collections can behave incorrectly.

Preparation focus. Understand the equals/hashCode contract conceptually; no need for advanced hash-table internals.

HashMap Key Semantics

Concept. HashMap uses a key to store and retrieve a value. Keys should have stable, consistent equals() and hashCode() behavior.

What you should know for the assessment. Know that putting a value for an equal existing key replaces the mapping for that key. Values may repeat; keys represent lookup identity.

```
Map<String,Integer> stock = new HashMap<>();
stock.put("A101", 5);
stock.put("A101", 7); // same key now maps to 7
```

Common traps. Do not confuse key uniqueness with value uniqueness.

Preparation focus. Practice get(), put(), containsKey() conceptually.

Generics

Concept. Generics provide compile-time type safety and reusable parameterized code.

What you should know for the assessment. Know basic syntax such as List<String>, generic methods/classes, and why generics reduce unsafe casts.

```
List<String> cities = new ArrayList<>();
```

Common traps. Generics do not mean any type can be inserted into a typed collection. List<String> should contain String values.

Preparation focus. Read short generic declarations and identify the allowed element type.

Generic Wildcards

Concept. A wildcard represents an unknown type. ? extends Number means Number or some unknown subtype of Number.

What you should know for the assessment. Understand the conceptual difference between ? extends T (read values as T safely) and ? super T (a lower bound). Milestone 2 focuses mainly on recognition.

```
List<? extends Number> nums = List.of(1, 2, 3);
```

Common traps. Do not describe ? extends Number as only subclasses excluding Number itself.

Preparation focus. Remember: extends = upper bound; do not overlearn variance theory for this assessment.

ListIterator

Concept. ListIterator is specialized for List and can move forward and backward through list elements.

What you should know for the assessment. Know its distinction from Iterator: ListIterator supports previous()/hasPrevious() and can modify a list during iteration through its supported methods.

```
ListIterator<String> it = list.listIterator();
while (it.hasNext()) { System.out.println(it.next()); }
```

Common traps. Do not modify a list structurally with unrelated list methods while iterating; use iterator-supported operations when required.

Preparation focus. Recognize when backward navigation is needed.

Iterator Removal

Concept. Iterator.remove() removes the last element returned by next() in a way coordinated with the iterator.

What you should know for the assessment. Know the safe pattern for conditional removal while iterating.

```
Iterator<Integer> it = nums.iterator();
while (it.hasNext()) {
    if (it.next() < 0) it.remove();
}
```

Common traps. Calling `collection.remove()` inside a for-each loop can cause concurrent modification problems.

Preparation focus. Be able to distinguish `iterator.remove()` from modifying the collection independently.

Numeric Type Promotion

Concept. Java promotes smaller numeric operands according to binary numeric promotion rules. For example, byte and short values are promoted to int in arithmetic expressions.

What you should know for the assessment. For assessment-level code, predict the type/result of simple arithmetic expressions involving int, long, float and double.

```
byte a = 10, b = 20;
int c = a + b; // result is int
```

Common traps. Do not assume `byte + byte` produces byte.

Preparation focus. Practice a few simple expressions; no need for advanced constant-expression edge cases.

Short-Circuit Logical Operators

Concept. `&&` stops if the left side is false; `||` stops if the left side is true. This can prevent the right-hand expression from executing.

What you should know for the assessment. Predict whether a right-hand expression is evaluated and understand why short-circuiting is useful for null checks and guarded conditions.

```
if (user != null && user.isActive()) { ... }
```

Common traps. `&` and `|` on booleans do not short-circuit in the same way.

Preparation focus. Trace left-to-right evaluation in simple conditions.

Operator Evaluation and Precedence

Concept. Operators have precedence rules that determine grouping before evaluation.

What you should know for the assessment. Know common precedence for arithmetic, comparison and logical operators. Use parentheses when clarity matters.

```
int x = 2 + 3 * 4; // 14
```

Common traps. Do not calculate strictly left-to-right without considering precedence.

Preparation focus. Practice reading expressions with `*`, `+`, relational and logical operators.

serialVersionUID

Concept. `serialVersionUID` is a version identifier used by Java serialization to verify compatibility between a serialized form and a class definition.

What you should know for the assessment. At Milestone 2 depth, know its purpose: serialization compatibility/version identification. You are not expected to design complex serialization versioning strategies.

```
private static final long serialVersionUID = 1L;
```

Common traps. It is not a database primary key, object id or security token.

Preparation focus. Know the one-line purpose and where it is declared.

Java Fundamentals - Practice Drill

1. Which runtime component executes Java bytecode?
2. When would you choose a Set instead of a List?
3. Why should a custom HashSet element implement equals() and hashCode() consistently?
4. What does List<? extends Number> mean?
5. Why is Iterator.remove() safer than directly removing from the collection during iteration?
6. What happens to the right side of false && expression?

Quick answer guide

1. The JVM executes Java bytecode.
2. Choose a Set when logical uniqueness is required and indexed duplicate-preserving order is not the main need.
3. Hash-based uniqueness uses both equality and hashing; inconsistent implementations can break logical duplicate handling.
4. The list element type is Number or an unknown subtype of Number.
5. The iterator coordinates structural modification with its traversal state.
6. It is not evaluated because && short-circuits when the left side is false.

2. Java Object-Oriented Programming

Assessment emphasis: 16 MCQs, all at Easy/Basic level. Expect code interpretation and concept comparison rather than framework-level design.

Classes, Objects and Constructors

Concept. A class defines structure and behavior; an object is an instance. Constructors initialize newly created objects.

What you should know for the assessment. Know default/no-argument vs parameterized constructors and that constructors do not have a return type.

```
class Product {
    String name;
    Product(String name) { this.name = name; }
}
```

Common traps. A method with the same name as a class and a return type is a method, not a constructor.

Preparation focus. Practice identifying which constructor runs for new Product(...).

Constructor Chaining, this() and super()

Concept. this() invokes another constructor in the same class; super() invokes a parent constructor. When used explicitly, the constructor call must be the first statement.

What you should know for the assessment. Know execution order at a beginner level: parent constructor runs before the child constructor body.

```
class B extends A {
    B() { super(); System.out.println("B"); }
}
```

Common traps. this() and super() cannot both be explicit first statements in the same constructor.

Preparation focus. Trace output order for two-level constructor examples.

Method Overloading

Concept. Overloading means same method name with different parameter lists, typically resolved at compile time.

What you should know for the assessment. Recognize legal overloads based on number/type/order of parameters.

```
void print(int x) {}
void print(String x) {}
```

Common traps. Changing only return type does not create a valid overload.

Preparation focus. Given two signatures, decide whether they are overloads.

Overload Resolution

Concept. The compiler selects the most applicable overloaded method based on argument types and conversions.

What you should know for the assessment. At this level, reason about exact matches vs simple widening conversions.

Common traps. Do not treat overloaded dispatch as runtime polymorphism.

Preparation focus. Practice with int vs long and Object vs String overloads.

Method Overriding and Dynamic Binding

Concept. Overriding occurs when a subclass provides an implementation of an inherited instance method with a compatible signature. Runtime dispatch selects the implementation based on the actual object.

What you should know for the assessment. Know that a parent reference can point to a child object and an overridden method call executes the child implementation.

```
Animal a = new Dog();
a.sound(); // Dog implementation if sound is overridden
```

Common traps. Static methods are hidden rather than overridden in the same polymorphic sense.

Preparation focus. Trace parent-reference/child-object examples.

Overriding Access Rules

Concept. An overriding method cannot reduce the visibility of the inherited method.

What you should know for the assessment. Know that public must remain public, while a protected method may be widened to public.

Common traps. Do not narrow public to protected/private when overriding.

Preparation focus. Focus on the simple visibility rule, not every checked-exception covariance edge case.

Runtime Polymorphism

Concept. Polymorphism allows a parent/interface reference to refer to different concrete implementations and invoke behavior appropriate to the actual object.

What you should know for the assessment. Understand why this reduces coupling and allows interchangeable implementations.

Preparation focus. Explain polymorphism using one interface and two implementations.

Inheritance

Concept. Inheritance models an is-a relationship and allows a subclass to reuse/extend accessible members of a parent.

What you should know for the assessment. Know the purpose of extends and basic parent-child relationships.

Common traps. Prefer composition when the relationship is has-a rather than is-a.

Preparation focus. Classify small examples as inheritance or composition.

Composition vs Inheritance

Concept. Composition builds behavior by containing/using another object, whereas inheritance derives from a parent type.

What you should know for the assessment. Use composition for has-a relationships and to reduce tight coupling when subclassing is not semantically correct.

Example. A Car has an Engine -> composition. A Dog is an Animal -> inheritance.

Preparation focus. Identify the relationship from the business statement.

Interfaces

Concept. An interface defines a contract that implementing classes agree to provide.

What you should know for the assessment. Know implements, multiple-interface implementation, and why interfaces support abstraction and interchangeable implementations.

```
interface Payable { double amount(); }
```

Common traps. An interface is not instantiated directly.

Preparation focus. Recognize when a common capability across unrelated classes fits an interface.

Abstract Classes

Concept. An abstract class can contain both implemented behavior/state and abstract methods. It cannot be instantiated directly.

What you should know for the assessment. Know the difference from an interface at conceptual level: abstract class can provide shared state/implementation and supports single class inheritance.

Preparation focus. Be able to choose abstract class when closely related subclasses share code/state.

Encapsulation

Concept. Encapsulation hides internal state and exposes controlled operations, commonly using private fields and public methods.

What you should know for the assessment. Understand why direct field access may violate invariants and why getters/setters or domain methods can validate updates.

```
private double balance;
public void deposit(double amt) { if (amt > 0) balance += amt; }
```

Preparation focus. Identify private state + controlled public behavior as encapsulation.

final Methods

Concept. A final method cannot be overridden by subclasses.

What you should know for the assessment. Know final on variable, method and class conceptually: no reassignment, no override, no inheritance respectively.

Common traps. final does not automatically make referenced objects immutable.

Preparation focus. Recognize the effect of final in short snippets.

this and super Keywords

Concept. this refers to the current object; super refers to the immediate parent portion of the current object.

What you should know for the assessment. Know common uses: resolve field shadowing, call current-class constructor, access parent implementation/constructor.

```
this.name = name;  
super.display();
```

Preparation focus. Map each keyword to current object vs parent class behavior.

SOLID - Single Responsibility Principle

Concept. SRP says a class should have one primary reason to change, meaning it should focus on one responsibility.

What you should know for the assessment. At Milestone 2 depth, identify an overly broad class and recognize splitting unrelated responsibilities as an improvement.

Example. An InvoiceCalculator should not also send email and write database backups.

Preparation focus. Think in terms of cohesive responsibility, not memorizing all SOLID definitions in depth.

Java OOP - Practice Drill

7. What is the key difference between overloading and overriding?
8. When a parent reference holds a child object, which implementation is chosen for an overridden instance method?
9. Why might composition be better than inheritance for a has-a relationship?
10. Can a final method be overridden?
11. What does SRP try to prevent?

Quick answer guide

1. Overloading uses the same name with different parameters and is resolved at compile time; overriding replaces inherited behavior and supports runtime dispatch.
2. The implementation belonging to the actual child object is invoked.
3. Composition models has-a relationships and often reduces unnecessary coupling.
4. No.
5. Classes that mix unrelated responsibilities and therefore have multiple independent reasons to change.

3. Java File I/O and Core APIs

Assessment emphasis: 12 MCQs. Four are basic recall and the rest require Basic Application or code interpretation. Practice small runnable snippets rather than memorizing large APIs.

Byte Streams vs Character Streams

Concept. Byte streams such as `FileInputStream/FileOutputStream` work with raw bytes. Character-oriented writers/readers are better suited to text.

What you should know for the assessment. Choose byte streams for binary data and character/text utilities for textual content.

Preparation focus. Classify image/PDF as binary and CSV/log text as character-oriented use cases.

FileOutputStream

Concept. `FileOutputStream` writes bytes to a file.

What you should know for the assessment. Know the purpose, create/overwrite vs append constructor concept, and the need to close resources.

```
try (FileOutputStream out = new FileOutputStream("data.bin")) {
    out.write(bytes);
}
```

Common traps. It does not format text like `PrintWriter`.

Preparation focus. Recognize it as a byte-output class.

PrintWriter

Concept. `PrintWriter` provides convenient formatted/text output methods such as `print()`, `println()` and `printf()`.

What you should know for the assessment. Know when it is more convenient than writing raw bytes for reports/text files.

```
try (PrintWriter pw = new PrintWriter("report.txt")) {
    pw.println("Completed");
}
```

Preparation focus. Match the API to textual formatted output.

Try-with-Resources

Concept. Try-with-resources automatically closes `AutoCloseable` resources when the block exits, including when exceptions occur.

What you should know for the assessment. Be able to recognize the pattern and explain why it reduces resource leaks.

```
try (BufferedReader br = Files.newBufferedReader(path)) {
    System.out.println(br.readLine());
}
```

Common traps. A resource declared outside the parentheses is not automatically managed in the same simple way.

Preparation focus. Use it for files, streams, JDBC connections/statements/result sets where appropriate.

Streams API

Concept. The Java Streams API processes collections declaratively through operations such as filter, map, sorted, reduce and collect.

What you should know for the assessment. Understand intermediate vs terminal operations conceptually and be able to read short pipelines.

```
List<String> names = users.stream()
    .filter(User::isActive)
    .map(User::getName)
    .toList();
```

Common traps. A Java Stream is not the same as a file InputStream or Spark Streaming.

Preparation focus. Trace filter -> map -> collect/toList for small datasets.

Regular Expressions: Pattern/Matcher

Concept. Regex defines a text pattern; Pattern compiles it and Matcher applies it to input.

What you should know for the assessment. Recognize simple validation/search patterns and distinguish full-string match from substring search.

```
Pattern p = Pattern.compile("[A-Z]{3}\\d{3}");
Matcher m = p.matcher("ABC123");
boolean ok = m.matches();
```

Preparation focus. Practice simple email/ID/phone-style recognition; no need for advanced lookarounds.

matches() vs find()

Concept. Matcher.matches() attempts to match the entire input; find() searches for the next matching subsequence.

What you should know for the assessment. Choose matches() for whole-value validation and find() for searching inside longer text.

Common traps. Do not treat both methods as equivalent.

Preparation focus. Use one-line examples where a pattern appears inside a sentence.

Java Date/Time API

Concept. java.time provides modern immutable date/time types such as LocalDate, LocalDateTime and ZonedDateTime.

What you should know for the assessment. Know which type includes a time zone and basic operations such as plusDays, parsing/formatting and comparison conceptually.

```
LocalDate today = LocalDate.now();
LocalDate due = today.plusDays(7);
```

Preparation focus. Match the type to the requirement: date only, date+time, or date+time+zone.

ZonedDateTime

Concept. ZonedDateTime represents date and time with a time-zone region/offset context.

What you should know for the assessment. Use it when the same event must be understood across geographic time zones.

```
ZonedDateTime now = ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));
```

Preparation focus. Know why LocalDateTime alone does not carry zone information.

HTTP Client API

Concept. java.net.http.HttpClient sends HTTP requests and receives responses.

What you should know for the assessment. At assessment depth, recognize HttpClient, HttpRequest and HttpResponse roles and basic GET flow.

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder(URL.create(url)).GET().build();
HttpResponse<String> res = client.send(req, HttpResponse.BodyHandlers.ofString());
```

Preparation focus. Understand request -> client.send -> response. No advanced async/network tuning required.

transient Fields

Concept. A transient field is skipped by default Java object serialization.

What you should know for the assessment. Know why sensitive/derived/non-serializable fields may be excluded from serialized state.

```
private transient String cachedToken;
```

Common traps. transient is about default serialization, not access control or database persistence.

Preparation focus. Recognize the effect on serialization.

Directory Traversal

Concept. Directory traversal visits files/subdirectories, commonly using java.nio.file.Files.walk or similar APIs.

What you should know for the assessment. Know the purpose and that streams/resources from traversal should be closed appropriately.

```
try (Stream<Path> paths = Files.walk(root)) {
    paths.filter(Files::isRegularFile).forEach(System.out::println);
}
```

Preparation focus. Understand use cases such as listing files, calculating sizes or finding extensions.

Java I/O and APIs - Practice Drill

12. Which Java I/O family is appropriate for raw binary image bytes?
13. Why is try-with-resources preferred for file and JDBC resources?
14. Which Matcher method is best for validating an entire ID string?
15. Which java.time type includes time-zone information?
16. What does transient do during default serialization?

Quick answer guide

1. Byte streams such as `FileInputStream/FileOutputStream`.
2. It automatically closes managed resources and reduces resource leaks.
3. `matches()`.
4. `ZonedDateTime`.
5. It excludes the field from the default serialized object state.

4. Java Concurrency, JDBC and Advanced Utilities - Stretch Preparation

Assessment emphasis: 8 MCQs, all marked Medium-High / Stretch. These topics are assessed conceptually. You should understand the problem each mechanism solves; production tuning and deep internals are not required.

Study strategy: Do not spend most of your revision time on API details here. Learn the failure mode (race, deadlock, connection overhead, rollback need, asynchronous dependency) and the matching concept.

Race Conditions and Synchronization

Concept. A race condition occurs when multiple threads access shared mutable state and the result depends on timing/interleaving.

What you should know for the assessment. Recognize lost updates and understand that synchronization/locks or thread-safe structures can protect a critical section.

Example. Two threads both read counter=10, both compute 11, both write 11. One increment is lost.

Preparation focus. Identify shared state + non-atomic read-modify-write as the warning sign.

Deadlocks and Lock Ordering

Concept. Deadlock can occur when threads wait forever for locks held by each other.

What you should know for the assessment. Know the classic circular-wait pattern and the simple prevention idea of acquiring multiple locks in a consistent global order.

Preparation focus. Given Thread A locks X then Y and Thread B locks Y then X, identify deadlock risk.

ConcurrentHashMap

Concept. ConcurrentHashMap is a Map implementation designed for safe concurrent access and updates without synchronizing the entire map externally for simple operations.

What you should know for the assessment. At this depth, choose it over HashMap when multiple threads need concurrent map access. Recognize that compound operations may require atomic map methods.

Preparation focus. Know the use case; no need for segmentation/bin-level implementation details.

CopyOnWriteArrayList

Concept. CopyOnWriteArrayList creates a fresh internal copy on modification, making iteration safe without external locking but making frequent writes expensive.

What you should know for the assessment. Choose it conceptually for read-heavy, write-rarely scenarios such as listener lists.

Common traps. It is not a good default for write-heavy workloads.

Preparation focus. Remember: many reads + rare writes.

CompletableFuture

Concept. CompletableFuture represents an asynchronous computation that can be chained or combined.

What you should know for the assessment. Understand basic purpose: run work asynchronously, transform results, or coordinate dependent tasks.

```
CompletableFuture.supplyAsync() -> loadData()
    .thenApply(data -> summarize(data));
```

Preparation focus. Recognize async composition conceptually; advanced chaining semantics are not required.

JDBC Transactions: Commit and Rollback

Concept. A transaction groups database changes so they succeed or fail as a unit. commit() makes changes permanent; rollback() undoes uncommitted changes.

What you should know for the assessment. Understand setAutoCommit(false), execute related updates, commit on success, rollback on failure.

```
conn.setAutoCommit(false);
try {
    // update A
    // update B
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
}
```

Preparation focus. For transfer/order scenarios, look for the all-or-nothing requirement.

HikariCP and Connection Pooling

Concept. Connection pooling reuses database connections instead of opening a new physical connection for every operation. HikariCP is a popular JDBC connection pool.

What you should know for the assessment. Know the performance/resource-management motivation and the correct lifecycle idea: borrow a connection, use it, close/return it to the pool.

Common traps. Do not keep borrowed connections forever; close() usually returns a pooled connection rather than destroying the pool.

Preparation focus. Conceptual recognition only; pool-size tuning is beyond this assessment.

Java Concurrency/JDBC - Stretch Drill

17. What symptom usually indicates a race condition?
18. How does consistent lock ordering help with deadlocks?
19. When is CopyOnWriteArrayList most suitable?
20. Why use a JDBC transaction for updating two related tables?
21. What problem does a connection pool solve?

Quick answer guide

1. The result changes depending on thread timing, often causing lost or inconsistent updates.
2. It removes inconsistent circular acquisition order, reducing circular wait.

3. Read-heavy workloads with infrequent writes.
4. To make both updates commit together or roll back together.
5. It avoids repeatedly creating expensive physical database connections by reusing them.

5. Scala Basics and Data Structures

Assessment emphasis: 12 MCQs, all Medium but at Basic Application depth. The focus is Scala syntax literacy needed to read and write Spark-oriented code.

val vs var

Concept. val creates a binding that cannot be reassigned; var creates a reassignable binding.

What you should know for the assessment. Know why val is preferred in functional/Scala style and that val does not automatically make a referenced mutable object deeply immutable.

```
val tax = 0.18
var attempts = 0
attempts += 1
```

Preparation focus. Identify which declarations can be reassigned.

Type Inference

Concept. Scala can infer many variable and expression types from assigned values.

What you should know for the assessment. Read declarations such as val total = 10 and understand the inferred type. Know when explicit types improve APIs/readability.

```
val name = "Asha" // String inferred
val score = 95    // Int inferred
```

Preparation focus. No need for compiler-level inference rules; focus on common literal/function examples.

Tuples

Concept. A tuple groups a fixed number of values that may have different types.

What you should know for the assessment. Know tuple creation, positional access and destructuring/pattern matching.

```
val order = (101, "Chennai", 2500.0)
val (id, city, amount) = order
```

Preparation focus. Use tuples for small grouped values; recognize _1, _2 or destructuring syntax.

Arrays

Concept. Scala Array is an indexed mutable sequence-like structure backed by JVM arrays.

What you should know for the assessment. Know indexed access and that array elements can be updated.

```
val a = Array(10, 20, 30)
a(1) = 25
```

Common traps. Do not confuse Array mutability with immutable List behavior.

Preparation focus. Compare Array vs List in one sentence.

Immutable Lists and Prepending

Concept. Scala List is an immutable linked list. The :: operator prepends an element and returns a new list.

What you should know for the assessment. Understand that original list remains unchanged.

```
val xs = List(2, 3)
val ys = 1 :: xs // List(1,2,3)
```

Preparation focus. Recognize immutable transformation rather than in-place mutation.

Immutable Collections

Concept. Scala favors immutable collection operations that return new collections instead of modifying existing ones.

What you should know for the assessment. Be able to reason about old vs new values after map/filter/prepend operations.

Preparation focus. Whenever you see val + immutable collection, assume operations return new values unless an explicitly mutable collection is used.

Pattern Matching

Concept. Pattern matching compares a value against ordered case patterns and can destructure values.

What you should know for the assessment. Understand simple match expressions for values, tuples and list shapes.

```
val label = status match {
  case 200 => "OK"
  case 404 => "Missing"
  case _ => "Other"
}
```

Preparation focus. Know case order and wildcard _ fallback.

Pattern Matching with Guards

Concept. A guard adds an if-condition to a case.

What you should know for the assessment. Understand that the pattern must match and the guard must be true.

```
x match {
  case n if n > 0 => "positive"
  case 0 => "zero"
  case _ => "negative"
}
```

Preparation focus. Trace simple guarded cases.

map Transformation

Concept. map applies a function to each element and returns one transformed element per input element.

What you should know for the assessment. Predict output shape and values for List.map.

```
List(1,2,3).map(_ * 10) // List(10,20,30)
```

Preparation focus. Remember one input element -> one output element.

Lambda Expressions

Concept. A lambda is an anonymous function value.

What you should know for the assessment. Read common syntax such as `x => x * 2` and underscore shorthand where unambiguous.

```
val double = (x: Int) => x * 2  
val out = List(1,2).map(double)
```

Preparation focus. Be able to identify parameter and returned expression.

Scala Basics - Practice Drill

- 22. What is the difference between `val` and `var`?
- 23. Does `1 :: xs` modify `xs`?
- 24. What does a pattern guard do?
- 25. What is the output cardinality relationship of `map`?
- 26. Which Scala structure in this chapter is naturally mutable: `Array` or immutable `List`?

Quick answer guide

- 1. `val` cannot be rebound; `var` can be reassigned.
- 2. No. It returns a new `List` with 1 prepended.
- 3. It adds a boolean condition that must be true for that case to match.
- 4. `map` produces one output element for each input element.
- 5. `Array`.

6. Scala Functional Transformations

Assessment emphasis: 8 Medium/Application questions. The focus is on reasoning through pipelines and recognizing functional-programming practices.

map vs flatMap

Concept. map returns one result per input element; flatMap maps and then flattens nested results into a single collection.

What you should know for the assessment. Use flatMap when one input can produce zero, one or many outputs, such as splitting sentences into words.

```
val lines = List("a b", "c")
lines.flatMap(_.split(" ")) // List("a","b","c")
```

Common traps. map(_.split(" ")) would produce a collection of arrays rather than one flat word collection.

Preparation focus. Practice nested collection examples.

filter and flatMap Pipelines

Concept. filter retains elements satisfying a predicate; flatMap expands remaining elements.

What you should know for the assessment. Read pipelines left-to-right and track the data shape after each stage.

```
records.filter(_.active).flatMap(_.contacts).filter(_.verified)
```

Preparation focus. Write a one-line description after each transformation.

Transformation Chaining

Concept. Scala transformations can be chained because each operation returns a new collection/value.

What you should know for the assessment. Understand order matters: filtering early can reduce work and change later results.

Preparation focus. Practice 3-step pipelines using filter, map, flatMap.

Higher-Order Functions

Concept. A higher-order function accepts a function as an argument or returns a function.

What you should know for the assessment. Recognize signatures such as $A \Rightarrow \text{Boolean}$ for predicates and $A \Rightarrow B$ for transformations.

```
def keep[A](xs: List[A], rule: A => Boolean) = xs.filter(rule)
```

Preparation focus. Identify the input and output types of simple function values.

Pure Functions

Concept. A pure function returns output determined only by its explicit inputs and does not create hidden side effects.

What you should know for the assessment. Prefer explicit inputs rather than reading mutable global state, clocks or external variables when testing business logic.

Example. `calculateTax(amount, rate)` is easier to reason about than `calculateTax(amount)` reading a mutable global rate.

Preparation focus. Recognize purity, determinism and lack of hidden mutation.

Immutability

Concept. Immutability means deriving new values instead of changing shared state in place.

What you should know for the assessment. Understand how immutable data supports safer reasoning, especially with concurrent/distributed processing.

Preparation focus. Given two designs, prefer returning a new collection/value over mutating a shared one when the scenario targets functional style.

Functional Pipelines

Concept. A functional pipeline expresses data processing as a sequence of composable transformations.

What you should know for the assessment. Be able to select `filter`/`map`/`flatMap` in the correct order for a simple requirement.

Preparation focus. Translate English requirements into a chain: `select` -> `transform` -> `expand` -> `aggregate` where appropriate.

Scala Transformations - Practice Drill

27. When should `flatMap` be preferred to `map`?
28. What function type represents a predicate from `A` to `Boolean`?
29. Why are pure functions easier to test?
30. What is the benefit of filtering before an expensive transformation when logically valid?

Quick answer guide

1. When one input may produce multiple/zero outputs and nested results should be flattened.
2. `A => Boolean`.
3. Their output depends only on explicit inputs and they avoid hidden side effects.
4. It reduces the number of elements processed by later expensive work.

7. Spark RDDs with Scala

Assessment emphasis: 16 MCQs, all Medium/Basic Application. This is one of the most important chapters. You must understand transformations, actions, lazy evaluation, lineage, pair RDDs, persistence and basic RDD creation.

Creating RDDs with parallelize

Concept. `SparkContext.parallelize` converts a local Scala collection into an RDD distributed across partitions.

What you should know for the assessment. Know the conceptual purpose and optional partition count.

```
val rdd = sc.parallelize(Seq(1,2,3,4), 2)
```

Common traps. `parallelize` starts from driver-resident local data and is mainly for small examples/testing.

Preparation focus. Recognize it as one RDD creation path.

Creating RDDs from External Sources

Concept. Spark can create RDDs from external storage such as text files using `SparkContext` APIs.

What you should know for the assessment. Know `textFile` as a basic example and distinguish it from `parallelize`.

```
val lines = sc.textFile("/path/orders.txt")
```

Preparation focus. Local collection -> `parallelize`; external text source -> `textFile`.

RDD Transformations

Concept. Transformations such as `map`, `filter`, `flatMap` and `reduceByKey` define a new RDD from an existing RDD.

What you should know for the assessment. Know that transformations are lazy and build lineage until an action requires execution.

```
val adults = people.filter(_.age >= 18)
```

Preparation focus. Classify common operations as transformation vs action.

Narrow Transformations

Concept. A narrow transformation can compute each output partition from a limited number of input partitions without broad redistribution, for example `map/filter` in typical cases.

What you should know for the assessment. Understand narrow vs wide conceptually; do not overfocus on scheduler internals.

Preparation focus. Recognize that group/repartition-style operations often require shuffle whereas `map/filter` typically do not.

RDD Actions

Concept. Actions trigger execution and return a result to the driver or write output, for example `count()`, `collect()`, `take()`, `reduce()` or save operations.

What you should know for the assessment. Know action examples and the consequence that `collect()` brings all elements to the driver.

```
val n = rdd.count()
val sample = rdd.take(5)
```

Preparation focus. Memorize a small set of common actions and understand `collect()` risk.

`collect()`

Concept. `collect()` returns the entire RDD as an array to the driver.

What you should know for the assessment. Use it only when data is known to be small enough for driver memory.

Common traps. Do not collect hundreds of GB to the driver. Prefer distributed writes/aggregations or small `take()` samples.

Preparation focus. In scenario questions, large dataset + `collect()` + driver OOM -> avoid `collect()`.

Lazy Evaluation

Concept. Spark delays transformation execution until an action is called.

What you should know for the assessment. Understand why lazy evaluation allows Spark to build an execution plan and avoid doing unused work.

Preparation focus. Given `map/filter` without an action, know the transformations are defined but not fully executed.

RDD Lineage

Concept. Lineage records the chain of transformations used to derive an RDD. Spark can use lineage to recompute lost partitions.

What you should know for the assessment. Know lineage as the logical dependency history that supports fault tolerance.

Preparation focus. Do not confuse lineage with checkpoint data itself.

Pair RDD Aggregations

Concept. Pair RDDs contain key-value tuples. `reduceByKey` combines values for the same key using an associative/commutative reduction function.

What you should know for the assessment. Understand basic per-key totals/counts.

```
val totals = sales.map(x => (x.region, x.amount)).reduceByKey(_ + _)
```

Preparation focus. Practice building (key,value) tuples before `reduceByKey`.

Pair RDD Operations and Joins

Concept. Pair RDD APIs support `join`, `groupByKey`, `reduceByKey`, `keys`, `values` and related operations.

What you should know for the assessment. Know `join` conceptually and understand that key-based combinations can cause shuffles.

```
val joined = orders.join(customers)
```

Preparation focus. Identify common key across both RDDs.

RDD Persistence

Concept. persist/cache keeps computed partitions for reuse so repeated actions can avoid recomputing the entire lineage.

What you should know for the assessment. Use persistence when the same expensive RDD is reused multiple times.

```
val cleaned = raw.filter(valid).persist()
cleaned.count()
cleaned.saveAsTextFile(out)
```

Common traps. Caching everything wastes memory. Persistence is useful when reuse justifies it.

Preparation focus. Look for reused expensive intermediate RDDs in scenarios.

Persistence Storage Levels

Concept. Storage levels control whether cached partitions are stored in memory, disk, serialized form, etc.

What you should know for the assessment. At this assessment depth, understand MEMORY_ONLY vs MEMORY_AND_DISK conceptually.

Preparation focus. If data may not fit in memory but should still be reused, MEMORY_AND_DISK is a common conceptual choice.

mapPartitions

Concept. mapPartitions invokes a function once per partition and gives access to an iterator for that partition.

What you should know for the assessment. Know the use case: reduce per-record setup overhead when expensive initialization can be shared within a partition.

Common traps. It is not the first transformation to use for ordinary element-wise logic; map is simpler for normal per-record work.

Preparation focus. Conceptual recognition only.

RDDs - Practice Drill

31. What is the difference between a transformation and an action?
32. Why can collect() be dangerous on a large RDD?
33. What does lineage provide?
34. When should an RDD be persisted?
35. Why is reduceByKey typically preferred for per-key aggregation rather than collecting values to the driver?
36. What problem can mapPartitions help reduce?

Quick answer guide

1. Transformations define new RDDs lazily; actions trigger execution and return/write results.
2. It transfers all records to the driver and can exhaust driver memory.
3. The dependency chain used to derive data, supporting recomputation after partition loss.
4. When an expensive RDD will be reused across multiple actions or downstream computations.

5. It performs distributed per-key aggregation instead of centralizing all data on the driver.
6. Repeated per-record initialization overhead by sharing setup within each partition.

8. Scala File I/O and Error-Handling Types

Assessment emphasis: 8 Medium/Basic Application questions. Focus on Source, proper closure, and the role of Try/Option/Either.

scala.io.Source

Concept. scala.io.Source is a basic Scala utility for reading text from files/URLs.

What you should know for the assessment. Know fromFile, getLines/mkString and the need to close Source after use.

```
val src = scala.io.Source.fromFile(path)
try {
  val text = src.mkString
} finally {
  src.close()
}
```

Common traps. Do not present Source.fromFile(...).mkString as complete production-style code without considering resource closure.

Preparation focus. Recognize Source for text input and close it reliably.

Reading Complete Text Content

Concept. mkString combines all characters/elements into one String.

What you should know for the assessment. Understand that it is convenient for small text files but not always ideal for huge files.

Preparation focus. Use getLines for line-oriented iteration and mkString for complete small-content reads.

Option

Concept. Option represents a value that may be present (Some) or absent (None), avoiding routine use of null.

What you should know for the assessment. Know Some/None and basic getOrElse/map usage conceptually.

```
val city: Option[String] = findCity(id)
println(city.getOrElse("Unknown"))
```

Preparation focus. Use Option when absence is expected and not necessarily an error.

Try, Success and Failure

Concept. Try represents a computation that may throw: Success(value) or Failure(exception).

What you should know for the assessment. Use it for exception-producing operations when you want a value-oriented result.

```
val n = scala.util.Try(text.toInt)
val safe = n.getOrElse(0)
```

Preparation focus. Know Try is about exception capture; Option is about presence/absence.

Try Recovery

Concept. recover can convert selected failures into fallback values.

What you should know for the assessment. Understand basic failure recovery without needing advanced combinators.

```
val value = Try(load()).recover { case _: IOException => defaultValue }
```

Preparation focus. Identify fallback behavior after an exception.

Either

Concept. Either represents one of two possible values, commonly Left(error) or Right(success).

What you should know for the assessment. Understand when richer error information is needed than Option provides.

Example. Right(order) for valid input, Left("invalid amount") for a validation error.

Preparation focus. Remember the common convention: Right = success, Left = error.

Scala File I/O - Practice Drill

- 37. Why should Source be closed?
- 38. When is Option more appropriate than Try?
- 39. What do Success and Failure represent?
- 40. Why might Either be preferred over Option for validation?

Quick answer guide

- 1. To release the underlying file/resource handle.
- 2. When absence is expected rather than caused by an exception.
- 3. The successful result or captured failure of an exception-producing computation.
- 4. Either can carry explicit error information in Left while success is carried in Right.

9. Spark DataFrames with Scala

Assessment emphasis: 8 Medium/Application questions. You must reason about schemas, structured expressions and when DataFrames are preferable to low-level RDDs.

RDD vs DataFrame Reasoning

Concept. RDDs provide low-level distributed collections. DataFrames provide schema-aware, column-oriented structured APIs that Spark can optimize more effectively.

What you should know for the assessment. For structured tabular/JSON/Parquet data and SQL-style operations, prefer DataFrames. RDDs are more appropriate for specialized unstructured/custom-object processing when structured APIs do not fit.

Preparation focus. In scenarios, ask whether the data has named fields/schema and whether filtering/aggregation/joining is SQL-like.

Schema Inference vs Explicit Schema

Concept. Schema inference asks Spark to inspect data and guess types; an explicit schema declares expected names/types/nullability.

What you should know for the assessment. Know why production-style ingestion often benefits from explicit schemas: consistency, predictable types and fewer surprises.

```
val schema = StructType(Seq(
  StructField("id", IntegerType, false),
  StructField("amount", DecimalType(10,2), true)
))
val df = spark.read.schema(schema).csv(path)
```

Preparation focus. Understand inferSchema convenience vs explicit-schema control.

Schema Handling and Malformed Values

Concept. A schema defines expected data types. Source data that does not conform may be parsed as null/corrupt or trigger behavior depending on reader mode/options.

What you should know for the assessment. At assessment depth, know that declaring a DecimalType does not magically convert arbitrary malformed text into a valid decimal.

Preparation focus. Look for the difference between schema declaration and actual data quality.

Structured Expressions vs RDD Functions

Concept. DataFrame expressions such as col(), filter(), groupBy() and built-in functions preserve structure/semantics that Spark can analyze and optimize.

What you should know for the assessment. Prefer built-in DataFrame/Spark SQL functions over opaque custom logic when possible.

```
df.filter(col("amount") > 1000).groupBy("region").sum("amount")
```

Preparation focus. Recognize optimization visibility as a key DataFrame advantage.

Scala in Batch Spark Jobs

Concept. Scala can be used to build batch Spark applications that read data, transform DataFrames and write results.

What you should know for the assessment. Know the overall flow: SparkSession -> read -> transform -> action/write.

```
val df = spark.read.parquet(in)
val out = df.filter($"active" === true)
out.write.mode("overwrite").parquet(dest)
```

Preparation focus. Be able to order pipeline stages correctly.

Spark DataFrames - Practice Drill

- 41. Why are DataFrames usually preferred for structured analytics?
- 42. What is the main benefit of an explicit schema?
- 43. Does DecimalType guarantee every source string is a valid decimal?
- 44. Why are built-in DataFrame functions often preferable to opaque custom logic?

Quick answer guide

- 1. They carry schema/column semantics and give Spark more opportunities to optimize structured operations.
- 2. Predictable data types and consistent ingestion behavior.
- 3. No. Bad input can still fail parsing or be treated as malformed/null depending on reader configuration.
- 4. Spark can understand and optimize built-in expressions more effectively.

10. Advanced Spark - Execution Plans, UDFs and External Sources

Assessment emphasis: 8 Medium/Application scenario questions. The goal is practical reasoning, not internal engine implementation.

Logical vs Physical Execution Plans

Concept. Spark builds logical plans for what the query means and physical plans for how Spark will execute it.

What you should know for the assessment. At assessment depth, know that `df.explain()` helps inspect plans and that `Exchange` often indicates data redistribution/shuffle.

```
df.groupBy("region").sum("amount").explain(true)
```

Preparation focus. Recognize execution plan as the right diagnostic representation for joins/aggregations/shuffles.

Exchange and Shuffle

Concept. An `Exchange` in a Spark physical plan commonly represents repartitioning/data movement required by operations such as some joins and aggregations.

What you should know for the assessment. Understand that shuffle is expensive compared with local partition processing and is often a performance consideration.

Preparation focus. No need to memorize every physical operator; identify redistribution conceptually.

Built-in Functions vs Custom UDFs

Concept. Built-in Spark SQL functions expose semantics to Spark and are generally easier to optimize than opaque custom UDF logic.

What you should know for the assessment. Prefer built-ins when an equivalent exists; use UDFs for custom logic not expressible conveniently with built-ins.

```
// Prefer built-in lower(), regexp_replace(), when(), etc. when suitable
```

Common traps. Do not use UDF as the default answer for every transformation.

Preparation focus. In scenarios, first ask: can a built-in function solve this?

Filter Before Expensive UDF

Concept. When valid logically, apply selective built-in filters before an expensive UDF so the UDF runs on fewer rows.

What you should know for the assessment. Understand this as a simple performance pattern.

Preparation focus. Look for a scenario where only 10% of rows need custom computation.

JDBC Integration and Parallel Reads

Concept. Spark JDBC can read database tables/queries. Partition options allow parallel reads when an appropriate numeric/date partition column and bounds are available.

What you should know for the assessment. Know `partitionColumn`, `lowerBound`, `upperBound` and `numPartitions` conceptually. `numPartitions` also limits concurrent JDBC partitions/connections.

Preparation focus. Avoid choosing excessive partitions beyond what the database can support.

Parquet Integration and Pruning

Concept. Parquet is columnar and works well with Spark. Selecting only needed columns enables column pruning; filtering partition columns can enable partition pruning.

What you should know for the assessment. Understand why reading only needed columns/partitions reduces I/O.

Preparation focus. Recognize Parquet + selective projection/filter as an efficient structured-data path.

JSON and Parquet Integration

Concept. JSON is flexible/semi-structured and often convenient for raw ingestion; Parquet is columnar and efficient for downstream analytics.

What you should know for the assessment. Understand a common raw-to-curated flow: ingest JSON, validate/transform, write Parquet/Delta-style structured output.

Preparation focus. Choose format based on stage and workload rather than assuming one is always best.

Advanced Spark - Practice Drill

45. What does `df.explain()` help you inspect?
46. What does an Exchange commonly indicate?
47. Why prefer built-in Spark SQL functions over a UDF when possible?
48. Why might you filter before applying an expensive UDF?
49. What is the purpose of `numPartitions` in a partitioned JDBC read?
50. What is column pruning in Parquet?

Quick answer guide

1. The logical/physical execution plan used for a DataFrame query.
2. Data redistribution/repartitioning, often a shuffle.
3. Built-ins expose semantics Spark can optimize and are usually more efficient/transparent.
4. To reduce the number of rows requiring the expensive custom function.
5. It controls the number of JDBC read partitions/concurrent tasks for the partitioned read.
6. Reading only the required columns instead of all columns.

11. Spark Streaming, Structured Streaming and Fault Tolerance

Assessment emphasis: 8 Medium-High / Stretch questions. Focus on conceptual behavior: DStreams vs Structured Streaming, checkpointing, windows, late data and source reliability. Deep state-store internals are outside the intended preparation depth.

DStreams vs Structured Streaming

Concept. DStreams are the older Spark Streaming abstraction based on micro-batches of RDDs. Structured Streaming uses the DataFrame/Dataset model and declarative structured APIs.

What you should know for the assessment. Know that new applications generally favor Structured Streaming and that DStreams remain relevant mainly for legacy understanding.

Preparation focus. Be able to identify which API matches DataFrame-style streaming.

Streaming from Files

Concept. Structured Streaming can treat a directory/table/source as an unbounded input and process newly arriving data incrementally.

What you should know for the assessment. Understand that files should appear atomically and checkpointing helps track progress across restarts.

Preparation focus. Simple scenario: new CSV/JSON files arrive throughout the day -> file streaming source.

Socket Streaming

Concept. Socket sources are commonly used for demonstrations because they are easy to start but are not durable production sources.

What you should know for the assessment. Know the distinction between a training/demo source and a replayable durable source.

Preparation focus. If a scenario asks for production recovery, do not choose an ephemeral socket source as the durability mechanism.

Checkpointing and Stateful Recovery

Concept. A checkpoint location stores progress/state metadata needed for a compatible streaming query to resume after failure.

What you should know for the assessment. Use durable/reliable storage for checkpoints and keep the same compatible query/checkpoint relationship for recovery.

Preparation focus. Question cue: restart without starting from zero or losing state -> checkpoint.

DStream Checkpointing

Concept. Legacy Spark Streaming can checkpoint metadata/state to reliable storage to support recovery and truncate long lineage for stateful operations.

What you should know for the assessment. Know the purpose and requirement for reliable storage. Do not memorize StreamingContext recovery APIs unless specifically taught.

Preparation focus. Differentiate DStream checkpoint concept from RDD cache/persist.

Windowed Operations

Concept. A window groups recent streaming records over a time interval, often with a slide interval controlling how frequently results update.

What you should know for the assessment. Understand window length vs slide interval conceptually.

Example. 30-minute window updated every 5 minutes.

Preparation focus. Read a business requirement such as rolling 10-minute sales totals and choose a window operation.

Late Data and Watermarks

Concept. Event-time processing may receive records after their event time. Watermarks express how long the engine should wait/retain state for late events in certain stateful operations.

What you should know for the assessment. At this depth, understand the trade-off: longer watermark tolerance accepts later data but may retain more state longer.

Preparation focus. No need for edge-case internals; focus on event time, lateness threshold and bounded state.

Streaming - Stretch Drill

51. Which API is generally preferred for new DataFrame-style Spark streaming applications?
52. Why should checkpoint data be stored durably?
53. What is the difference between window length and slide interval?
54. What problem do watermarks help manage?
55. Why is a socket source usually considered a demo source?

Quick answer guide

1. Structured Streaming.
2. So progress/state metadata survives process/driver failure and the query can recover.
3. Window length is the amount of event history included; slide interval is how often the window result advances/updates.
4. Late event handling and bounded retention of state for event-time operations.
5. It is ephemeral and generally not designed for durable replay/recovery like production messaging/file sources.

12. Final Assessment Preparation Plan

Use the following plan in the final revision cycle. It is designed around the actual depth represented in the finalized Milestone 2 question bank.

Stage	Recommended work	Evidence you are ready
1. Java Core	Revise JVM, stack/heap, collections, generics, operators, iterators.	You can answer basic concept/code questions without running the program.
2. Java OOP	Trace constructors, overloading/overriding, polymorphism, interfaces, composition, final/this/super.	You can explain why the selected method/constructor runs.
3. Java APIs	Run small I/O, regex, date/time, stream and HTTP snippets.	You can select the right API from a simple business requirement.
4. Scala Basics	Practice val/var, tuples, List/Array, match, guards, lambdas.	You can predict short Scala snippet outputs.
5. Scala Functional	Practice map/filter/flatMap, pure functions and HOF signatures.	You can translate a requirement into a 2-4 stage pipeline.
6. RDDs	Practice parallelize/textFile, map/filter, actions, reduceByKey, join, cache/persist.	You can classify transformation/action and explain lazy evaluation/lineage.
7. DataFrames	Review explicit schema, RDD vs DataFrame, built-ins.	You choose DataFrames for structured SQL-style processing and explain why.
8. Advanced Spark	Practice explain(), UDF-vs-built-in, JDBC partition read concepts, Parquet pruning.	You can identify the performance mechanism in a short scenario.
9. Streaming	Review checkpoint, window, watermark, source durability.	You can identify the correct recovery/window concept without deep API syntax.
10. Stretch Java	Review race/deadlock, transaction rollback, concurrent utilities, pooling.	You can map each failure scenario to the correct concept.

Rapid Comparison Sheet

Pair	Key distinction
Overloading vs Overriding	Overloading: same name, different parameters, compile-time choice. Overriding: subclass replacement, runtime dispatch.
List vs Set vs Map	List: ordered sequence/duplicates. Set: uniqueness. Map:

	key-value lookup.
val vs var	val cannot be rebound; var can be reassigned.
Array vs List	Array elements are mutable/indexed; List is immutable and optimized for head/tail style operations.
map vs flatMap	map keeps one result per input; flatMap expands and flattens nested outputs.
Transformation vs Action	Transformation builds lazy lineage; action triggers Spark execution.
RDD vs DataFrame	RDD is low-level distributed collection; DataFrame is schema-aware and optimizer-friendly for structured data.
cache/persist vs checkpoint	Persist speeds reuse of computed data; checkpoint stores recovery/progress/state metadata depending on Spark context.
DStream vs Structured Streaming	DStream is legacy RDD-based streaming; Structured Streaming uses DataFrame/Dataset structured APIs.
matches vs find	matches validates the whole input; find locates a matching subsequence.
commit vs rollback	commit makes transaction changes permanent; rollback undoes uncommitted work.
Race vs Deadlock	Race: timing-dependent incorrect shared-state result. Deadlock: threads wait indefinitely for each other.

Trainee Readiness Checklist

- ☐ I can explain JVM, bytecode, stack and heap using a simple example.
- ☐ I can choose List, Set or Map for a business requirement.
- ☐ I can distinguish overloading from overriding and trace runtime polymorphism.
- ☐ I can explain this, super, final, composition and SRP.
- ☐ I can identify safe file/JDBC resource management using try-with-resources.
- ☐ I can read simple Java Stream, regex and java.time code.
- ☐ I can explain val/var, tuples, immutable List, Array and pattern matching.
- ☐ I can predict map/filter/flatMap pipelines and recognize pure functions.
- ☐ I can classify Spark transformations and actions and explain lazy evaluation and lineage.
- ☐ I can use reduceByKey/join conceptually for pair RDDs and explain why collect() can be dangerous.
- ☐ I can explain when to persist an RDD and the purpose of storage levels.
- ☐ I can explain RDD vs DataFrame and explicit vs inferred schema.
- ☐ I can identify when built-in Spark functions are preferable to UDFs.
- ☐ I can interpret explain()/Exchange conceptually and recognize shuffle/data redistribution.

- ☐ I can explain partitioned JDBC reads and Parquet column/partition pruning.
- ☐ I can explain DStreams vs Structured Streaming, windows, watermarks and checkpointing.
- ☐ I can recognize race conditions, deadlocks and the purpose of concurrent collections.
- ☐ I can explain JDBC transaction commit/rollback and connection pooling at conceptual level.

Suggested 48-Hour Revision Sequence

Time Block	Focus
Day 1 - Block 1	Java fundamentals + collections + OOP
Day 1 - Block 2	Java I/O/APIs + short runnable exercises
Day 1 - Block 3	Scala basics + functional transformations
Day 2 - Block 1	RDDs + pair RDDs + persistence + lineage
Day 2 - Block 2	DataFrames + schemas + Advanced Spark scenarios
Day 2 - Block 3	Streaming + JDBC/concurrency stretch revision + full self-check

Final advice: Do not memorize answer positions or wording from practice banks. Prepare the underlying concept well enough that you can explain why the correct option solves the scenario and why the alternatives do not.

Scope note: This handbook follows the topics, subtopics and depth represented in the finalized Databricks Milestone 2 V5.1 question bank. It intentionally keeps advanced concurrency/streaming/JDBC topics at conceptual/stretch depth rather than production-engineering depth.